

FPGA–SN65HVD230–ELM327 CAN/OBD-II Debug Report

Workspace: /home/andy/verilog_workspace/assume_board_is_nexys3

Final successful test: 2026-06-07, Asia/Taipei

1 Executive Summary

The system is now working. The FPGA, through an SN65HVD230 CAN transceiver, responds to an ELM327 adapter on 11-bit, 500 kbit/s OBD-II CAN. The final successful ELM327 test returned:

```
0100: 7E806410000180000
010C: 7E804410C2EE0
010D: 7E803410D58
```

The implemented dummy ECU behavior is:

Request	Response	Meaning
0100	7E8 06 41 00 00 18 00 00	PID 0C and PID 0D supported
010C	7E8 04 41 0C 2E E0	RPM = $0x2EE0/4 = 3000$ RPM
010D	7E8 03 41 0D 58	Speed = $0x58 = 88$ km/h

The root cause was hardware wiring: CAN high/low were misplaced relative to the SN65HVD230/ELM327 bus. Before the fix, the ELM side could produce differential activity, but the SN65 RXD input into the FPGA did not see valid CAN edges. After correcting the high/low placement, the passive FPGA listener changed from idle-only output to activity output, and the active OBD responder successfully answered the ELM327.

2 Current System Behavior

2.1 Protocol and IDs

The active bitstream currently programs the FPGA as a small OBD-II ECU:

- CAN bitrate: 500 kbit/s.
- CAN frame format: standard 11-bit data frames.
- Accepted request IDs: functional $0x7DF$ and physical $0x7E0$.
- Response ID: $0x7E8$.
- Accepted service: mode 01 current data.
- Supported PIDs: 00, 0C, and 0D.

2.2 Debug UART Behavior

The CP2102 debug UART is 9600 baud. Its FPGA output is intentionally inverted in the top level:

```
assign uart_tx_cp2102_inv = ~dbg_uart_tx;
```

This inversion is required by the current wiring/debug path.
The active OBD node prints two kinds of debug messages:

Heartbeat H<rx><edge><sof><ack><ok><crc_err><stuff><att><tx_ack><done><busy> once per second.

OBD request R<pid_byte> when a supported OBD request is decoded. The PID byte is raw binary, so PID 00, 0C, and 0D may not print as ordinary visible characters.

Heartbeat fields are:

Field	Meaning
rx	Current SN65 RXD logic level seen by FPGA
edge	Any CAN RX edge since last heartbeat
sof	CAN receiver saw start-of-frame
ack	CAN receiver reached ACK slot while receiving a frame
ok	CRC-valid standard data frame received
crc_err	CRC or format rejection seen
stuff	Bit stuffing error seen
att	FPGA attempted a transmit since last heartbeat
tx_ack	Another CAN node ACKed the FPGA transmit
done	FPGA transmit completed
busy	CAN controller is currently busy

During the final successful run, the CP2102 log included:

```
H10000000000
R<00>
R<0C>
R<0D>
H11111001110
H10000000100
```

The important heartbeat was H11111001110: RX edges were seen, SOF was seen, the receiver reached ACK slot, CRC was OK, a transmit was attempted, the transmit was ACKed by the ELM327, and transmit completed.

3 Final Wiring

3.1 Pin and Signal Map

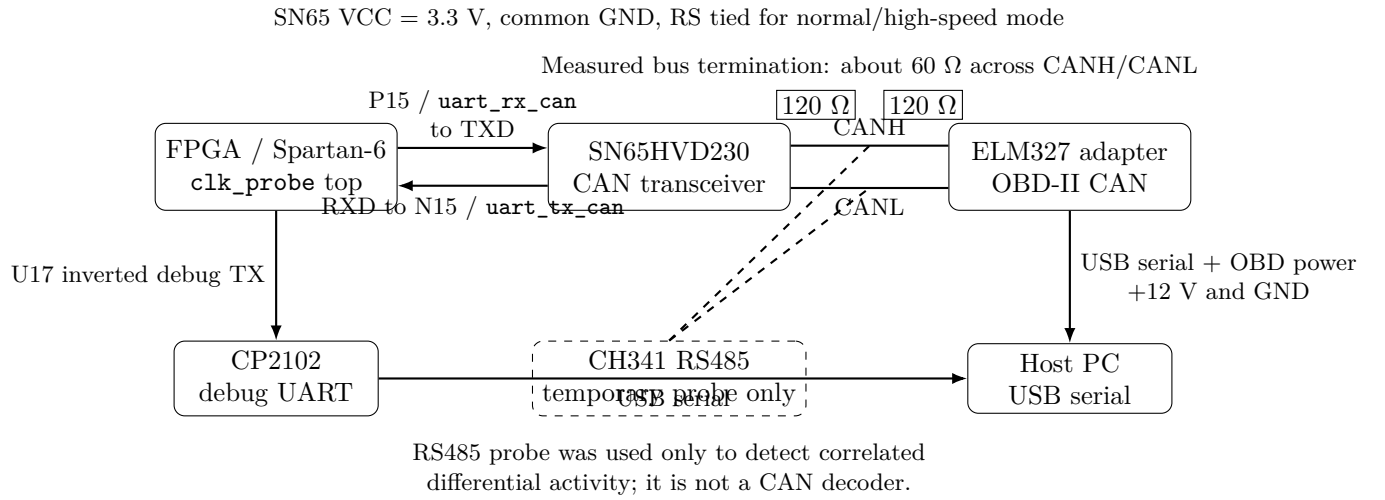
The names `uart_tx_can` and `uart_rx_can` are historical/misleading. The verified final mapping is:

FPGA pin	Verilog net	Direction at FPGA	External connection
U10	clk_in	input	100 MHz board clock
N15	uart_tx_can	input	SN65HVD230 RXD
P15	uart_rx_can	output	SN65HVD230 TXD
U17	uart_tx_cp2102_inv	output	CP2102 RX/debug UART, inverted

FPGA pin	Verilog net	Direction at FPGA	External connection
P16	uart_rx_cp2102	input	CP2102 TX, currently unused by OBD node
D11	slow_blink_clk_led	output	slow heartbeat LED
C11	unused_led	output	held low
T18	unused_out	output	held low

The B8 direct CANL probe was removed. It was not a useful final debug method because CANH/CANL are differential analog bus lines, and connecting one side directly to a digital FPGA input does not meaningfully decode the CAN bus.

3.2 TikZ Wiring Diagram



4 Source File Manual

File	Purpose
index.v	Top-level FPGA module <code>clk_probe</code> . Instantiates <code>obd_can_node</code> , maps SN65 RXD/TXD, preserves the required CP2102 debug inversion, drives LEDs/unused outputs. Edit this file when changing top-level wiring or replacing the active OBD node with a temporary debug module.
blink.ucf	Xilinx UCF pin constraints. Current CAN pins are N15 = SN65 RXD input and P15 = SN65 TXD output. B8 is intentionally absent. Edit this file only when physical pin assignments change.
obd_can_node.v	OBD-II application layer. Filters request IDs 0x7DF/0x7E0, recognizes service 01, supports PIDs 00, 0C, and 0D, and formats responses from ID 0x7E8. Also generates CP2102 debug output.

File	Purpose
<code>can_simple_controller.v</code>	Simple standard-frame CAN controller. Handles bit timing, receive parsing, CRC, bit stuffing, ACK slot drive, transmit frame preparation, and transmit ACK detection. Edit this carefully; most physical/debug issues should be isolated before changing this file.
<code>uart_tx.v</code>	UART transmitter used by debug output. Current debug baud is 9600. The top-level inverts its output for the CP2102 path.
<code>doall.sh</code>	Build/program script. Sources Xilinx ISE 14.7 settings, opens <code>assume_board_is_nexys3.xise</code> , runs <code>synthesize/implement/bitgen</code> , writes <code>burn.cmd</code> , and programs <code>clk_probe.bit</code> with iMPACT.
<code>cp2102workspace/cp2102_recv_uart_serial.py</code>	Receives serial port and prints debug UART characters. Use for CP2102 debug output.
<code>cp2102workspace/cp2102_send_bytes.py</code>	Send bytes that repeatedly sends one packed byte over serial. It is not needed for the current OBD responder flow.
<code>assume_board_is_nexys3.xise</code>	Xilinx ISE project file used by <code>doall.sh</code> .
<code>clk_probe.bit</code>	Generated bitstream that is programmed to the FPGA.

5 Build and Run Manual

5.1 Find the Serial Ports

USB serial numbering can change. Check stable symlinks first:

```
ls -l /dev/serial/by-id /dev/serial/by-path
ls -l /dev/ttyUSB* /dev/ttyACM*
```

During the successful run, the mapping was:

Device	Observed port	Serial settings
ELM327 / FTDI	<code>/dev/ttyUSB0</code>	38400 baud
CP2102 debug UART	<code>/dev/ttyUSB2</code>	9600 baud
Digilent/JTAG serial	<code>/dev/ttyUSB3</code>	not used for OBD test
CH341 RS485 probe	<code>/dev/ttyUSB1</code> when connected	500000 baud for activity probe only

If permission is needed, run commands through the `dialout` group, as was done during debug:

```
sg dialout -c "<command>"
```

5.2 Compile and Program the FPGA

The normal build/program command is:

```
cd /home/andy/verilog_workspace/assume_board_is_nexys3
sg dialout -c "./doall.sh"
```

`doall.sh` performs these operations:

1. Source `/opt/Xilinx/14.7/ISE_DS/settings64.sh`.
2. Open `assume_board_is_nexys3.xise` with `xtclsh`.
3. Run `Synthesize - XST`.

4. Run Implement Design.
5. Run Generate Programming File to create `clk_probe.bit`.
6. Create `burn.cmd`.
7. Program the FPGA through Digilent/iMPACT.

A successful program operation ends with text similar to:

```
INFO:iMPACT:188 - '1': Programming completed successfully.
'1': Programmed successfully.
```

5.3 Monitor FPGA Debug Output

Start CP2102 debug capture:

```
sg dialout -c "python3 cp2102workspace/cp2102_recv_uart.py --port /dev/ttyUSB2 --baud
9600"
```

Idle active-firmware output should look like:

```
H100000000000
H100000000000
```

5.4 Run the ELM327 OBD Test

The manual ELM command sequence is:

```
ATZ
ATE0
ATL0
ATSO
ATH1
ATSP6
ATCAF1
ATRV
0100
010C
010D
ATBD
```

A Python version used during the successful test was:

```
import serial, time
ser = serial.Serial('/dev/ttyUSB0', 38400, timeout=0.25)
time.sleep(0.1)
ser.reset_input_buffer()

def send(cmd, wait=2.0):
    ser.reset_input_buffer()
    ser.write((cmd + '\r').encode())
    end = time.time() + wait
    data = bytearray()
    while time.time() < end:
        b = ser.read(1)
        if b:
            data.extend(b)
```

```

        if b == b'>':
            break
    print(cmd + ': ' + data.decode(errors='replace').replace('\r', '\\r').replace('\n', '\\n'))

for c in ['ATZ', 'ATE0', 'ATL0', 'ATS0', 'ATH1', 'ATSP6', 'ATCAF1',
         'ATRV', '0100', '010C', '010D', 'ATCS', 'ATBD']:
    send(c, 2.5)
ser.close()

```

Expected successful response:

```

ATRV: 11.8V
0100: 7E806410000180000
010C: 7E804410C2EE0
010D: 7E803410D58
ATBD: 0C 00 00 07 E8 03 41 0D 58 00 00 00 00

```

6 What Was Wrong

The failure was not primarily the Verilog OBD response data. The key problem was physical CAN wiring: CANH/CANL placement was wrong at some point in the chain, so the SN65HVD230 side did not see the ELM327 bus correctly.

The strongest evidence was the combination of these observations:

- Before the final fix, ELM commands such as 0100, 010C, and 010D returned `CAN ERROR`.
- A passive FPGA/SN65 listener repeatedly printed R10000, meaning SN65 RXD stayed high and no edges/lows were observed.
- The CH341 RS485 differential probe saw bursts correlated with ELM requests, proving some differential activity existed on the ELM/probe pair.
- At the same time, the FPGA/SN65 RXD did not see that activity, proving the SN65/FPGA path was not on the same valid CAN signal polarity/pair.
- A temporary FPGA pulse through SN65 TXD was seen locally on SN65 RXD but not by the CH341 probe, again pointing to a bus-side wiring mismatch rather than only a Verilog parser issue.

After correcting the high/low placement, the passive listener changed to R111FF during an ELM request window. That meant RXD saw edges and dominant lows. Then the active responder immediately produced valid ELM responses.

7 Debug Timeline and Attempts

1. Read the existing project notes in `doing_stuff.txt` and verified that `blink.ucf` was the authoritative pin source.
2. Reconfirmed the final FPGA/SN65 pin interpretation: N15 is CAN RX into FPGA from SN65 RXD, and P15 is CAN TX from FPGA to SN65 TXD.
3. Reconnected a loose CAN TX pin. This did not yet make ELM requests work.
4. Provided ELM327 OBD-side 12 V and ground. ATRV then showed about 11.8V, but requests still returned `CAN ERROR`.

5. Added the second 120 Ω CAN terminator. The bus then measured about 60 Ω across CAN-H/CANL, which is the expected value for two 120 Ω terminators in parallel.
6. Tried CANH/CANL swaps and wire adjustments. Earlier attempts still did not produce valid SN65 RXD activity.
7. Programmed a passive raw listener that held FPGA CAN TX recessive forever. This answered the question of whether the FPGA/FFmpeg-like logic could listen without sending: yes, the FPGA can listen passively through the SN65 RXD pin, but before the hardware fix it saw no ELM activity.
8. Added a direct B8-to-CANL debug probe and a UCF pin for it. This was later removed because a single-ended digital FPGA input is not a meaningful CAN differential receiver.
9. Added a CH341 RS485 adapter as a temporary differential activity probe. It was useful only as a rough edge/activity detector, not as a CAN decoder. At 500000 baud it saw byte bursts correlated with ELM request timing.
10. Programmed a one-millisecond-per-second SN65 dominant pulse diagnostic. This proved the local FPGA/SN65 TXD/RXD side could react, but the pulse was not seen by the CH341 probe, strengthening the bus-side wiring diagnosis.
11. Corrected the CANH/CANL placement in hardware.
12. Reprogrammed the passive listener with B8 removed and CAN TX held recessive. It changed from R10000 idle/no activity to R111FF during ELM request activity.
13. Reprogrammed the active OBD responder, disabled the old periodic test TX, and verified successful ELM responses for 0100, 010C, and 010D.

8 Current Code Configuration

The final source state is intentionally simple:

- `index.v` instantiates `obd_can_node` directly.
- `blink.ucf` has 8 constrained I/O pins and no B8 probe.
- `obd_can_node.v` has `TEST_TX_ENABLE = 1'b0`, so it does not send periodic diagnostic frames.
- The FPGA only transmits when a supported OBD request is decoded, or when the CAN controller drives the ACK slot for a valid received frame.

9 Practical Notes for Future Work

- If ELM requests return `CAN_ERROR` again, first run the passive listener style test or inspect the active heartbeat. If there are no RX edges, debug hardware before changing the CAN parser.
- Keep the CP2102 debug inversion in `index.v`. Removing it causes unreadable debug output with the current wiring.
- Use `/dev/serial/by-id` to identify USB serial adapters. The `/dev/ttyUSBx` numbers changed during debug.

- Do not use the CH341 RS485 adapter as a CAN decoder. It can show correlated activity, but CAN framing, arbitration, stuffing, CRC, and ACK are not RS485 UART protocol.
- Keep CAN termination at two $120\ \Omega$ endpoints. A powered-down or disconnected setup should still measure about $60\ \Omega$ across CANH/CANL when both terminators are present.
- If adding more PIDs, edit the supported PID mask and response case in `obd_can_node.v`. The current dummy data is deliberately fixed.